

RTL Design Sherpa

HIVE Micro-Architecture Specification 0.2

October 13, 2026

Table of Contents

1 Hive Mas Index....11

2 Document Information....12

2.1 HIVE Micro-Architecture Specification....12

2.2 Document Purpose....12

2.3 References....12

2.4 Terminology....13

2.5 Revision History....13

3 Design Philosophy and Machine Organization....15

3.1 Philosophy....15

3.2 Machine Organization....15

3.3 What Is Imported vs. What Is Built....16

4 Clocking and Reset....17

4.1 Clock Domains....17

4.2 CDC Structure....17

4.3 Reset Architecture....18

4.4 Reset Sequencing....19

4.5 Soft Reset....19

4.6 Clock Gating (Optional)....19

5 hive_top....20

5.1 Composition....20

5.2 Structural Rules....20

6 hive_c_controller (Block 1)....22

6.1 Submodules....22
6.2 Interfaces....23
6.3 Decisions (owner, 2026-10-13)....23
6.4 Open Items....23
7 serv_monitor (Block 2)....24
7.1 Submodules....24
7.2 Interfaces....25
7.3 Open Items....25
8 control_network (Block 3)....26
8.1 Submodules....26
8.2 Performance Verification Hooks....27
9 config_manager (Block 4)....28
9.1 Submodules....28
9.2 Interfaces....29
10 Common Utilities....30
11 Internal Packages and Wiring....31
11.1 Package Organization....31
11.2 Key Internal Interfaces....31
11.3 Wiring Conventions....32
12 The Descriptor Injection Cycle, Step by Step....33
13 The Monitoring Cycle, Step by Step....34
13.1 Steady State (every serv_clk cycle)....34
13.2 Periodic Report....34
13.3 Congestion Alert (urgent path)....34

13.4 Error Path....34
14 The Context Switch Sequence, Cycle by Cycle....36
15 Reset and Boot Behavior....38
16 Verification Hooks and Observability....39
16.1 Observation Bus....39
16.2 Protocol Checkers (immediate assertions)....39
16.3 Formal Properties....39
16.4 Testbench Structure (cocotb, rds-dv framework)....40
17 Parameters, Defines, and Filelists....41
17.1 Hardware Parameters (includes/hive_config.svh)....41
17.2 Software Knobs (sw/common/hive_config.h)....41
17.3 Planned Filelist Layout....41
17.4 Scaling Knobs....42
18 HIVE Hardware Architecture Specification Index....43
18.1 Document Organization....43
18.1.1 Front Matter....43
18.1.2 Chapter 1: Introduction....43
18.1.3 Chapter 2: System Overview....43
18.1.4 Chapter 3: Subsystem Architecture....43
18.1.5 Chapter 4: Interfaces....43
18.1.6 Chapter 5: Programming Model....43
18.1.7 Chapter 6: Performance....44
18.1.8 Chapter 7: Integration and Verification....44
18.2 Related Documentation....44

19 Product Requirements Document (PRD)....45
19.1 HIVE - Hierarchical Intelligent Vector Environment....45
19.2 1. Executive Summary....45
19.2.1 1.1 Quick Stats....45
19.2.2 1.2 Subsystem Goals....45
19.3 2. Documentation Structure....45
19.3.1 HIVE Hardware Architecture Specification (HAS) — external contract....45
19.3.2 HIVE Micro-Architecture Specification (MAS) — implementation intent....46
19.3.3 Other Documentation....46
19.4 3. Architecture Overview....46
19.4.1 3.1 Top-Level Block Diagram....46
19.4.2 3.2 Resource Budget (NexysA7 100T)....47
19.5 4. Key Features....48
19.5.1 4.1 Hierarchical RISC-V Architecture....48
19.5.2 4.2 Distributed Monitoring....49
19.5.3 4.3 Network Reconfiguration....49
19.5.4 4.4 RAPIDS Integration....49
19.6 5. Interfaces....50
19.6.1 5.1 External Interfaces....50
19.6.2 5.2 Virtual Tile Mapping....50
19.7 6. Use Cases....50
19.7.1 6.1 Descriptor Scheduling....50

19.7.2 6.2 Congestion Management....51
19.7.3 6.3 Network Reconfiguration....51
19.8 7. Test Coverage....51
19.8.1 7.1 Current Status....51
19.9 8. Performance Targets....51
19.9.1 8.1 Latency Targets....51
19.9.2 8.2 Monitoring Overhead....52
19.10 9. Integration Guidelines....52
19.10.1 9.1 HIVE + RAPIDS + Delta Integration....52
19.11 10. Development Status....53
19.11.1 10.1 Current Phase....53
19.11.2 10.2 Roadmap....53
19.12 11. Educational Value....53
19.13 13. Documentation Generation....54
19.13.1 13.1 Generating PDF/DOCX from the Specifications....54
19.14 14. References....54
19.14.1 14.1 Internal Documentation....54
19.14.2 14.2 Related Subsystems....54
19.14.3 14.3 External References....54
19.15 Navigation....55
20 RAPIDS Beats Architecture MAS....56
20.1 Document Organization....56
20.1.1 Front Matter....56
20.1.2 Chapter 1: Overview....56

20.1.3 Chapter 2: FUB (Functional Unit Blocks)....56
20.1.4 Chapter 3: Macro (Integration Blocks)....56
20.1.5 Chapter 4: Interfaces....57
20.2 Quick Reference....57
20.2.1 FUB Modules (fub_beats/)....57
20.2.2 Macro Modules (macro_beats/)....58
20.3 Architecture Comparison: RAPIDS vs STREAM....59
20.4 Clock and Reset Summary....60
20.4.1 Clock Domains....60
20.4.2 Reset Signals....60
20.5 Interface Summary....61
20.5.1 External Interfaces....61
20.5.2 Internal Buses....61
20.6 Related Documentation....61
20.7 Specification Conventions....62
20.7.1 Signal Naming....62
20.7.2 Parameter Naming....62
20.7.3 Figure and Table Numbering....62
21 Delta Micro-Architecture Specification Index....63
21.1 Overview....63
21.1.1 Key Features....63
21.1.2 Companion Specifications....63
21.2 Navigation....64
21.2.1 Front Matter....64

21.2.2 Chapter 1: Introduction....64
21.2.3 Chapter 2: Block Descriptions....64
21.2.4 Chapter 3: Cycle-Level Behavior....64
21.2.5 Chapter 4: Generated RTL....64
21.2.6 Chapter 5: Verification....64
22 RISC-V Falcon Suite....65
22.1 The ladder....65
22.1.1 Adjacent: hive-serv....66
22.2 Goals per rung....66
22.3 Packaging (per rung, the MC-line treatment)....67
22.4 References and layout....68

List of Figures

No figures in this document.

List of Tables

Table 1: Reference Documents....12
Table 2: Terminology and Definitions....13
Table 3: Document Revision History....13
Table 4: Imported vs. built....16
Table 5: Clock domains....17
Table 6: Domain resets....18
Table 7: Subordinate instances....20
Table 8: Utility inventory....30
Table 9: Package layout....31
Table 10: Checker inventory....38
Table 11: FUB Module Summary....56
Table 12: Macro Module Summary....57
Table 13: RAPIDS Beats vs STREAM Comparison....58
Table 14: Clock Domains....59
Table 15: Reset Signals....59
Table 16: External Interfaces....60
Table 17: Internal Buses....60

1 Hive Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 HIVE Micro-Architecture Specification

Document Number: HIVE-MAS-001 **Version:** 0.2 **Status:** Draft (pre-RTL — implementation intent)
Classification: Open Source - MIT License

2.2 Document Purpose

This Micro-Architecture Specification (MAS) defines the intended internal implementation of the HIVE subsystem at block level: the module hierarchy, the internal interfaces between blocks, the cycle-level behavior of the key operations, the verification hooks the RTL will expose, and the configuration surface (parameters, defines, filelists). It is written for the RTL implementer and for verification engineers building block-level testbenches.

The MAS assumes the external contracts of the companion HAS — packet formats, register map, interface classes — and does not restate them except where an internal design decision depends on the boundary detail.

Status note: no RTL exists yet. Everything in this document is implementation intent to seed RTL development; where a decision is still open, it is marked as an open item rather than resolved silently.

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	HIVE HAS v0.2 (`docs/hive_has/`)	Hardware Architecture Specification (external contract)
[REF-2]	HIVE PRD (`PRD.md`)	Product requirements and roadmap
[REF-3]	VexRiscv documentation (https://github.com/SpinalHDL/VexRiscv)	HIVE-C core generator; plugin/config reference
[REF-4]	SERV documentation (https://github.com/olofk/serv)	Monitor core; RF/WB interfaces
[REF-5]	Delta MAS	Router/NI internals HIVE configures

ID	Document	Description
	(`../../../../../fabric-gen-ip/delta/docs/delta_mas/delta_mas_index.md`)	and monitors
[REF-6]	Repo CDC library (`rtl/cdc/`)	Async FIFO / synchronizer building blocks for the four clock domains

2.4 Terminology

Terminology and Definitions

Term	Definition
FUB	Functional Unit Block — a leaf RTL module
Context memory	BRAM holding the four routing contexts per tile
Injection FIFO	Per-source AXIS TX buffering ahead of the Delta ingress
Quiesce	Drain-and-hold state a router enters before a context switch
Tap	Read-only observation port from a router into its SERV monitor
Wrapper	SV module that instantiates an external core (VexRiscv/SERV) plus glue

2.5 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-12	seang	Initial MAS release; migrates the implementation content of the retired v0.3 `hive_specification.md` into block-level implementation intent
0.2	2026-10-13	seang	Owner decisions: quiesce

Version	Date	Author	Description
			status via fabric IDLE signal into a PeakRDL config register (hard interlock demoted to optional hardening); kestrel loader pattern; dual-flop tap synchronizers; descriptor mailbox parameter-controlled, dedicated register file default, custom-instruction alternative rejected; PKT_CONFIG injection path still TBD. Addendum: delta is the fabric (owner); everything that leaves a block is a stream — control network AXIS-shaped (32-bit, `tuser[0]` urgent), no AXI4 full in HIVE

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Design Philosophy and Machine Organization

3.1 Philosophy

HIVE's micro-architecture follows three rules, in priority order:

1. **Hierarchy mirrors control.** Every block's structure reflects the control hierarchy: one `hive_c_controller` at the top, sixteen identical `serv_monitor` instances below it, one `control_network` binding them, one `config_manager` orchestrating the fabric they all touch. A reader should be able to draw the HAS block diagram from the RTL tree alone.

2. **Reuse the repo's fabric.** Clock crossing uses `rtl/cdc/` primitives; AXIS buffering uses the repo GAXI FIFO/skid library; arbitration uses the repo round-robin arbiter. HIVE's own RTL is only what is genuinely new: descriptor generation, monitor glue, control framing, context switching.

3. **Observability is a feature, not a debug afterthought.** Every block exposes the signals the Ocelus monitor system (see `rtl/amba/ocelus/`) and the cocotb testbenches need, on dedicated observation ports that are part of the module interface — never tapped ad hoc from internals.

4. **Everything that leaves a block is a stream** (owner, 2026-10-13). All fabric-facing traffic is AXI4-Stream; the internal control network is AXIS-shaped (32-bit frames on `tvalid/tready/tlast`, `tuser[0]` = urgent flag). Only register files are memory-mapped (AXI4-Lite). No AXI4 full anywhere in HIVE — it never touches DDR and moves no burst traffic. Rationale: delta speaks AXIS natively; inband descriptor injection only exists on the stream path; and the repo's stream verification stack (GAXI BFMs, Ocelus monitors, CDC FIFOs) attaches everywhere without bespoke protocol work.

> **Delta relationship (owner, 2026-10-13):** delta is the AXIS fabric generator and is the fabric; the archived NoC-mesh draft describes the same fabric (the re-home to `fabric-gen-ip/` was organizational). HIVE RTL implements against delta's AXIS interfaces exactly as the HAS specifies.

3.2 Machine Organization

```
hive_top                                # top-level integration
├── hive_c_controller/                  # Block 1
├── serv_monitor/          (16×)        # Block 2
├── control_network/                    # Block 3
├── config_manager/                     # Block 4
└── common/                             # shared utilities (or repo
imports)
```

Block details are specified in Chapter 2. The four clock domains and their crossings are specified in the next section of this chapter; CDC structure constrains several block interfaces, so it comes first.

3.3 What Is Imported vs. What Is Built

Imported vs. built

Item	Source	Why
HIVE-C CPU	VexRiscv (SpinalHDL, generated)	RV32IM 5-stage core; not reinvented
SERV monitor CPU	SERV (olofk, RTL)	Bit-serial RV32I; smallest monitor that can run real trigger logic
AXIS async FIFOs	repo GAXI / `rtl/common`	Gray-pointer CDC, already verified
Reset synchronizers	`rtl/cdc/`	Async-assert/sync-deassert, house style
Descriptors, taps, triggers, framing, contexts	**HIVE RTL (new)**	The actual product of this component

Last Updated: 2026-10-13

4 Clocking and Reset

4.1 Clock Domains

Clock domains

Domain	Clock	Frequency	Components
1	`hive_c_clk`	100 MHz (PoC), 150+ MHz (optimized)	VexRiscv core, I/D memories, descriptor generation, control-network master, CDA AXIS master
2	`serv_clk`	50-100 MHz	All 16 SERV cores, traffic counters, trigger logic, injection FIFOs, control-network slaves
3	`delta_clk`	100 MHz	Delta routers, NI, router configuration registers (Delta-owned; HIVE writes them)
4	`host_clk`	50-100 MHz, external	UART, AXI4-Lite slave, command parser

`serv_clk` is shared by all 16 monitors — one domain, sixteen instances. `delta_clk` may be phase-aligned with `hive_c_clk` in the PoC but must not be assumed synchronous; all crossings use the async FIFOs below regardless.

4.2 CDC Structure

Every crossing is a repo-library dual-clock FIFO or synchronizer; there are no combinational crossings.

```
hive_c_clk → delta_clk     CDA descriptors:      256-bit × 8 entries
delta_clk → hive_c_clk     PKT_STATUS return:    128-bit × 16 entries
(aggregates all SERVs)
serv_clk → delta_clk       Per-SERV injection:   128-bit × 4 entries ×
16
delta_clk → serv_clk       Per-SERV config in:   128-bit × 4 entries ×
16
host_clk → hive_c_clk      AXIL data:            32-bit × 8 entries;
control: dual-flop sync
hive_c_clk → serv_clk      Control commands:     32-bit × 2 entries per
```

```
SERV (low bandwidth)
serv_clk → hive_c_clk     Status uplink:        32-bit × 4 entries per
SERV, round-robin arbitrated
```

: CDC inventory

FIFO depths follow the house rule: ≥ 4 entries minimum to bound full/empty oscillation; deeper where aggregation justifies it (status return holds a full round of 16 SERV reports).

4.3 Reset Architecture

Global reset (`system_rst_n`): active-low, asynchronous assertion, synchronous deassertion to each domain. Minimum assertion: 16 cycles of the slowest clock (`serv_clk`).

Domain resets, each synchronized from `system_rst_n`:

Domain resets

Reset	Domain	Scope	Notes
`hive_c_rst_n`	hive_c_clk	VexRiscv, memories, control master	Memory init required after reset
`serv_rst_n[15:0]`	serv_clk	Individual monitors	Independently controllable: debug, recovery, reconfig
`delta_rst_n`	delta_clk	Routers, NI, config logic	Must drain in-flight packets first (asserted by config manager, not the host)
`host_rst_n`	host_clk	UART, AXIL slave, parser	Async to HIVE-C; commands require handshake

Reset synchronizer (all domains, house pattern — async assert, 2-stage sync release):

```
module reset_synchronizer (
    input  logic clk,
    input  logic async_rst_n,
    output logic sync_rst_n
);
    logic [1:0] sync_chain;
```

```
    always_ff @(posedge clk or negedge async_rst_n) begin
        if (!async_rst_n) sync_chain <= 2'b00;
        else              sync_chain <= {sync_chain[0], 1'b1};
    end
    assign sync_rst_n = sync_chain[1];
endmodule
```

4.4 Reset Sequencing

Assertion: `system_rst_n` low → all domain resets assert asynchronously → FSMs to IDLE/RESET → FIFO pointers clear → config registers to reset values.

Deassertion (staged): host interface → HIVE-C → Delta Network → SERV monitors (last). After deassert, HIVE-C loads firmware, Delta enters Context 0 (systolic), SERVs enter monitoring mode; system-ready after ~100 cycles of the slowest clock. See Chapter 4.4 for the boot-time behavior description.

4.5 Soft Reset

The soft-reset register (HAS §4.3) maps to `serv_rst_n[15:0]`, `delta_rst_n`, control-network reset, and `hive_c_rst_n` respectively. Soft resets are synchronous to the target domain and re-use the same staged sequencing; a HIVE-C self-reset (bit 0) is legal but leaves the network unsupervised until firmware re-initializes — firmware sets a watchdog policy for it.

4.6 Clock Gating (Optional)

- SERV gating: gate `serv_clk` per monitor when idle; ~5% system power; 1-2 cycle wake-up. Not in the PoC bring-up plan.
- Router gating: per Delta's own power plan (Delta MAS); HIVE only asserts tile-disable via PKT_CONFIG.
- HIVE-C gating: rejected — must stay responsive to interrupts.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 hive_top

Top-level integration of the four blocks. `hive_top` owns the external port list (HAS Chapter 4), the parameter surface (Chapter 6 of this book), and the instantiation of everything below. It contains no logic of its own beyond parameter propagation and the reset staging tree.

5.1 Composition

Subordinate instances

Instance	Module	Count	Role
`u_hive_c`	`hive_c_controller`	1	Master controller (Block 1)
`u_serv[k]`	`serv_monitor`	`SERV_COUNT` (16)	Per-tile monitors (Block 2)
`u_ctrl_net`	`control_network`	1	HIVE-C ↔️ SERV star (Block 3)
`u_cfg_mgr`	`config_manager`	1	Contexts and switching (Block 4)

5.2 Structural Rules

- `hive_top` does not create clocks; it takes the four domain clocks and `system_rst_n` from the system top (PLL/BUFG decisions live there).
- Every CDC FIFO listed in §1.2 is instantiated in the *sending* block and documented in that block's interface section; `hive_top` only routes the clock/reset pairs to them.
- The Delta-facing AXIS ports of `hive_c_controller` and each `serv_monitor` cross into `delta_clk` inside the sender, so `hive_top` presents only `delta_clk`-domain AXIS toward Delta — Delta never sees a foreign clock.
- All monitor observation ports (Ocelus tap-outs, per Chapter 5) are exported up through `hive_top` as a flat observability bus so system integration can attach the monitor system without touching block internals.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 hive_c_controller (Block 1)

The master controller: VexRiscv “Small” (RV32IM) plus the glue that turns firmware decisions into CDA packets, control-network frames, and register access. Clock domain: `hive_c_clk`.

```
hive_c_controller
├── vexriscv_wrapper.sv      # VexRiscv instantiation + debug/irq
plumbing
├── instruction_mem.sv       # 32 KB tightly-coupled I-mem (BRAM)
├── data_mem.sv              # 32 KB tightly-coupled D-mem (BRAM)
├── descriptor_gen.sv        # CDA descriptor assembly + injection
queue
├── axis4_master.sv          # AXIS TX (CDA) into delta_clk CDC FIFO
├── ctrl_net_master.sv       # Control-network command serializer
├── axil_master.sv           # AXI4-Lite master window
(RAPIDS/Delta/SERV cfg)
└── host_if.sv               # UART / AXI4-Lite slave command
interface
```

: HIVE-C module tree

6.1 Submodules

`vexriscv_wrapper`. Instantiates the generated VexRiscv core with the “Small” configuration. Exposes JTAG or UART-based firmware load (open item: loader transport — the falcon suite’s kestrel loader pattern is the preferred precedent), machine timer for the statistics interval, and the interrupt aggregation from `interrupt.c`’s hardware sources (RAPIDS completion, control-network alert, host command).

`instruction_mem` / `data_mem`. 32 KB each, single-cycle tightly-coupled BRAM. Dual-ported where the loader needs a debug port; the core port always wins arbitration.

`descriptor_gen`. Turns a firmware-written descriptor record (written into a small command mailbox in data-memory space) into a 256-bit CDA beat: field assembly per HAS §4.2, injection-FIFO push, and backpressure hold on `tready`. Holds up to `MAX_INFLIGHT_DESCRIPTORS` outstanding descriptors; full is a firmware-visible stall flag, not an error.

`axis4_master`. Drives the CDA AXIS interface into the 256×8 async FIFO that crosses to `delta_clk`. Asserts `tuser=2'b01`, `tdest=16`, `tlast` every beat. Single outstanding packet at a time (one descriptor = one beat).

`ctrl_net_master`. Serializes control-network commands from firmware: command + target mask + payload into 32-bit frames, multicast capable for broadcasts (`CONFIG_PREPARE`, `SET_ROUTING_MODE`, threshold updates).

`axil_master`. The 0x4000_0000 register window (HAS §4.3). Address decode for the RAPIDS, Delta, SERV-status, and config regions; a lightweight request/response queue so firmware never stalls on AXIL handshake.

`host_if`. UART and/or AXI4-Lite slave command channel (synthesis option). Normalizes both to the same internal command FIFO consumed by firmware.

6.2 Interfaces

External: host interface, CDA AXIS master (into CDC), control-network master, AXIL master.
Internal: memory buses (core-local), mailbox from firmware view, interrupt lines in.

6.3 Decisions (owner, 2026-10-13)

- **Firmware load transport: kestrel loader pattern.** HIVE-C adopts the rung-1 board-loader approach — an AXIL-slave loader with on-chip memories and load-then-run control, in the style of `kestrel_mem_loader` (see the kestrel MAS, `ch02_blocks/06_kestrel_mem_loader.md`). JTAG and preloaded-BRAM options are dropped.
- **Descriptor mailbox: parameter-controlled, dedicated register file is the default.** A synthesis parameter selects between a dedicated descriptor-register file and a data-memory aperture; RTL implements the register-file path first. The legacy spec's "custom instruction for atomic descriptor injection" alternative was considered and rejected — the mailbox + `descriptor_gen` model stands (policy stays in firmware; no VexRiscv plugin surgery).

6.4 Open Items

None in this block.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 serv_monitor (Block 2)

One per Delta tile; sixteen instances. The SERV core runs the trigger logic in firmware; the surrounding glue provides the counters, taps, and injection path the core alone cannot. Clock domain: `serv_clk` (all instances share the domain, never communicate directly).

```
serv_monitor
├── serv_wrapper.sv        # SERV core instantiation + bus glue
├── local_mem.sv           # 2 KB I-mem + 2 KB D-mem
├── traffic_counter.sv     # Packet/cycle counters from the router tap
├── trigger_logic.sv       # Threshold compare + event detect
(independent of core)
├── axis_injector.sv       # AXIS TX (PKT_CONFIG / PKT_STATUS) into
CDC FIFO
└── ctrl_net_slave.sv      # Control-network slave + mailbox to core
```

: SERV monitor module tree

7.1 Submodules

`serv_wrapper`. Instantiates the SERV core. Exposes a read-only tile-ID register at core address `0xF000_0000` (firmware reads its own identity) and routes the data bus to local memory, the injection FIFO aperture, the counter bank, and the control mailbox.

`traffic_counter`. The counter bank behind the router tap: `pkt_rx_count[4]`, `pkt_tx_count[4]`, `pkt_local_inject`, `pkt_local_consume`, `buffer_overflow_count`, `stall_cycles` — 32-bit wrapping, saturating-clear on read when `Reset Statistics Counters` is issued. `buffer_occupancy[4]`, `congestion_flag`, and `error_flags` are wired combinationally from the tap for low-latency trigger evaluation.

`trigger_logic`. Hardware threshold comparator running in parallel with the core: flags `occupancy > cfg_threshold` (default 75% of buffer depth) and error conditions immediately, so congestion detection meets the < 10 cycle target even when the bit-serial core is mid-instruction. The core consumes trigger flags as interrupts/polled events and decides the packet response; `trigger_logic` never injects packets by itself (firmware owns all packet content — this split keeps policy in software, detection in hardware).

`axis_injector`. TX FIFO aperture (128-bit × 4 entries) plus the AXIS master into the per-SERV `serv_clk → delta_clk` CDC FIFO. Firmware writes words, then commits with `TVALID`; the injector appends `tid`, `tuser`, `tdest`, `tlast` per the commit record.

`ctrl_net_slave`. Receives 32-bit control frames addressed to this monitor (HAS Chapter 4 command set), updates threshold/config registers, raises core interrupts, and queues uplink status frames for the control network's round-robin arbiter.

7.2 Interfaces

Router tap in (from the tile router, `delta_clk` — synchronized at the tap boundary inside `traffic_counter`; the tap is observation-only), control-network slave port, AXIS TX into CDC, observation port out.

7.3 Open Items

- Tap synchronization (decided 2026-10-13): **dual-flop synchronizers** per direction at the tap boundary inside `traffic_counter`.
- Whether `trigger_logic` also owns the periodic-report timer or the core does (core timer saves logic, hardware timer saves instruction budget — Phase 1 experiment).

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 control_network (Block 3)

The star between HIVE-C and the sixteen monitors. Low bandwidth, hard latency and fairness requirements: commands in < 10 cycles, uplink aggregation across all 16 < 20 cycles, no slave starves. The block spans two clock domains — master side in `hive_c_clk`, slave side in `serv_clk` — with the CDC FIFOs below as the only crossing. Per the everything-is-a-stream rule (§1.1), all ports are AXIS-shaped: 32-bit data beats with `tvalid/tready/tlast`, and `tuser[0]` carrying the urgent flag — the repo stream BFMs and Ocelus monitors cover this block exactly like the fabric streams.

```
control_network
├── control_packetizer.sv   # Command/status frame encode + decode
├── control_router.sv       # Star fan-out: address decode, multicast
├── control_arbiter.sv      # Round-robin uplink arbitration (16 → 1)
└── cdc_fifo_* (repo lib)   # Per-direction async FIFOs (see §1.2
depths)
```

: Control network module tree

8.1 Submodules

`control_packetizer`. Frames are 32-bit (HAS §4.1 contract), carried as the `tdata` payload of an AXIS beat (one frame per beat, `tlast` on every beat):

```
Bits [31:24]: Command / status opcode
Bits [23:16]: Target SERV id (command) or source SERV id (status);
              8'hFF = broadcast (commands only)
Bits [15:0]:  Payload (threshold values, context id, error code, ...)
```

Command and status share the opcode space with direction implied by the port. Multicast is mask-free: broadcast reaches all slaves; selective multicast is done by issuing multiple unicast frames.

`control_router`. Pure fan-out — one master port to sixteen slave ports, address-decoded, no buffering beyond the CDC FIFOs. In-order per slave; commands to different slaves may be in flight simultaneously.

`control_arbiter`. Round-robin over the sixteen uplink FIFOs, non-preemptive per frame; `tuser[0]` (urgent) is honored as next-grant priority (an alert frame marked urgent wins the grant after the current frame). Fairness is structural: equal weights, one frame per grant; starvation is impossible by construction and asserted formally.

8.2 Performance Verification Hooks

Grant sequence, per-slave service interval, and FIFO occupancy are exported to the observation bus; the < 20-cycle all-16 aggregation target is checked by a cocotb scoreboard counting grant cycles across a full round.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 config_manager (Block 4)

Owns the four virtual routing contexts and the atomic-switch orchestration. Lives in `hive_c_clk` with a `delta_clk` register interface; the PKT_CONFIG broadcast path carries context changes to the routers.

```
config_manager
├── config_registers.sv     # Mode select, status, soft-reset, error
sticky
├── context_memory.sv       # 4 contexts × per-tile routing tables
(BRAM)
└── switch_controller.sv    # PREPARE / SET_ROUTING_MODE / ACTIVATE
FSM
```

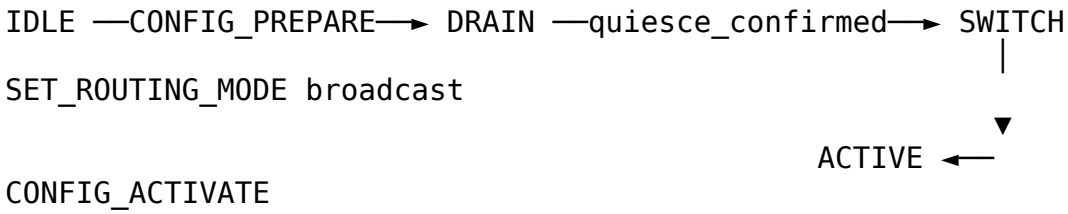
: Config manager module tree

9.1 Submodules

`context_memory`. Four contexts × 16 tiles × routing-table entry (BRAM; 2 BRAM blocks in the budget). Contexts 0-2 hold hardwired-mode metadata only (the routing functions are hardwired in Delta's routers); Context 3 holds full per-tile tables via `ROUTE_TABLE_n` writes. Reset default: Context 0 loaded/selected.

`config_registers`. The HIVE-side configuration space (HAS §4.3): active context, per-context valid bits, switch-status RO bits, error sticky flags, soft-reset outputs. Exposed to HIVE-C firmware over the AXIL window and to the observation bus.

`switch_controller`. The three-phase FSM (HAS §5.4):



- Quiesce status via IDLE register (decided 2026-10-13): an **IDLE signal** from the fabric — an aggregated quiescent indication covering the participating routers — is routed into a **PeakRDL-managed configuration register** for reading by HIVE-C firmware (and the host). Firmware verifies IDLE before issuing `SET_ROUTING_MODE`. The register bank follows the repo's PeakRDL convention (`rdl/` sources generating regmap, decode, and docs, as in `projects/components/legacy-ip/retro_legacy_blocks/rdl/`). A hard interlock that refuses the mode update in hardware remains optional future hardening, not a Phase-3 gate.

- `DRAIN`: broadcasts prepare, then polls the IDLE register until all participating routers report idle. Timeout → error sticky, abort.
- `SWITCH`: emits one PKT_CONFIG broadcast (command 0x0000_0001, payload = context id, tile mask = all). The broadcast is a single packet consumed by all routers in the same cycle — that simultaneity is what makes the switch atomic.
- Injection request path remains **TBD** (owner, 2026-10-13): whether PKT_CONFIG broadcasts ride the HIVE-C injection path or a SERV flood.

9.2 Interfaces

AXIL register window (from `hive_c_controller`), PKT_CONFIG injection request to HIVE-C's CDA/config injection path (the manager does not own an AXIS master; it borrows the HIVE-C injection path or a SERV flood — implementation decision in Phase 2), observation bus out.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Common Utilities

Shared helpers used by more than one block. Rule: a utility lives in HIVE's `common/` only when the repo library has no equivalent; otherwise it is imported, not copied.

Utility inventory

Module	Source	Used by	Notes
`gaxi_fifo_async` / AXIS async FIFO	repo GAXI library (`rtl/common`)	All CDC paths	Gray-pointer, house-verified; depths per §1.2
`reset_synchronizer`	`rtl/cdc/`	Every domain	Pattern in §1.3
`round_robin_arbiter`	repo common	`control_network`	Generic N-way; N=16 for the uplink
`credit_counter`	repo common (Delta-shared semantics)	Injection path	Credit accounting if Delta ingress requires it (confirm against Delta MAS)
`axis_fifo` (sync)	repo GAXI	Injection FIFOs	`serv_monitor` TX aperture

Parameter consistency: `SERV_COUNT`, buffer depths, and FIFO widths come from the single `hive_config.svh` include (Chapter 6); no block hardcodes its own copy.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Internal Packages and Wiring

11.1 Package Organization

One package per block plus a top-level config include; no cross-imports from leaf packages upward:

Package layout

File	Contents
`includes/hive_config.svh`	All `parameter` defaults (Chapter 6); single source of truth
`includes/hive_ctrl_net_pkg.sv`	Control-network opcodes, frame struct, urgent-flag encoding
`includes/hive_desc_pkg.sv`	CDA descriptor struct, field masks, `PKT_*` encodings
`includes/hive_cfg_pkg.sv`	Context ids, config register offsets, error codes
`includes/hive_obs_pkg.sv`	Observation-bus record layout (Chapter 5)

Structs (not raw bit vectors) cross block boundaries for every interface wider than a few signals — descriptor records, control frames, observation records — so port lists stay readable and compile-time checked.

11.2 Key Internal Interfaces

Descriptor mailbox → `descriptor_gen` (in `hive_c_controller`): firmware writes a `descriptor_t` record; `descriptor_gen` pulses `done` on handshake and raises `stall` while the injection queue is full.

Control network frame (§2.4): the 32-bit frame is the only master-to-slave and slave-to-master payload, carried on AXIS handshake signals (`tvalid/tready/tlast`, `tuser[0]` = urgent) per the everything-is-a-stream rule (§1.1); widening to 64-bit is an open area item if the statistics payload overflows 16 bits regularly.

Config-manager injection request: `switch_controller` raises `inject_req` with a completed PKT_CONFIG record; the HIVE-C injection path (or SERV flood, Phase-2 decision) carries it into Delta.

Observation bus (Chapter 5): a flat, `hive_c_clk`-domain record stream exported through `hive_top`; each block drives its slice; slices are concatenated, not arbitrated (fixed layout in `hive_obs_pkg`).

11.3 Wiring Conventions

- AXIS signals are named `axis_<role>_<signal>` with `role` ∈ {cda, stat, cfg, inj}; AXI4-Lite as `axil_<signal>`.
- Resets are `*_rst_n`, active low, synchronized per §1.3; no block generates its own reset from `system_rst_n` directly.
- Every inter-block interface documents its clock domain at the port; a port pair in different domains must name the CDC element in the block header comment.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 The Descriptor Injection Cycle, Step by Step

The defining HIVE-C operation: one descriptor from firmware decision to RAPIDS FIFO. Target: < 30 cycles `hive_c_clk` → RAPIDS accept.

```
Cycle  Action
  1    Firmware completes a descriptor_t write to the mailbox;
doorbell pulse.
  2    descriptor_gen pops the record; field assembly (HAS §4.2);
tuser/tdest/last fixed.
  3    Record pushed to injection queue; queue occupancy +1
(observation).
 4-6   Injection queue at head; CDC FIFO (hive_c_clk side) accepts;
tvalid/tready handshake.
 7-10  Gray-pointer cross to delta_clk side; FIFO output registered.
11-20  Delta traversal toward virtual tile 16 (router hops; ~2/hop,
edge-attached).
21-24  RAPIDS ingress handshake; descriptor written to the descriptor
FIFO.
       RAPIDS descriptor engine acks (tready) — injection latency
target met.
```

Ordering: descriptors leave `descriptor_gen` in mailbox order; the CDC FIFO and Delta preserve that order per stream, which is the ordering property the formal inventory asserts (descriptor FIFO ordering, HAS §7).

Backpressure: if RAPIDS' descriptor FIFO fills, `tready` deasserts and the CDC FIFO absorbs the stall; firmware sees `stall` only when the injection queue itself fills (`MAX_INFLIGHT_DESCRIPTORS` outstanding) — a policy signal to the scheduler, never a protocol error.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 The Monitoring Cycle, Step by Step

One SERV monitor's steady-state loop and its alert path.

13.1 Steady State (every `serv_clk` cycle)

1. `traffic_counter` samples the router tap: per-direction packet strobes increment `pkt_rx/tx_count`; occupancy strobes update `buffer_occupancy`.
2. `trigger_logic` compares occupancy against `cfg_threshold`; the congestion line is combinationally current with the tap (this is what meets the < 10-cycle detection target — no firmware round trip).
3. The SERV core polls (or is interrupted by) `trigger_logic` and the counter bank; on the periodic-report interval it snapshots metrics into the uplink frame.

13.2 Periodic Report

1. Core writes a status record into `ctrl_net_slave`'s uplink mailbox.
2. `control_arbiter` grants this slave's turn (round-robin; bounded wait ≤ 16 frames).
3. Frame crosses `serv_clk → hive_c_clk` (4-entry FIFO), is consumed by `ctrl_net_master`, and lands in the SERV-status AXIL window for firmware.

13.3 Congestion Alert (urgent path)

```
occupancy > threshold        (combinational, tap-current)
  → trigger_logic flags core (≤ 1 cycle)
  → core assembles PKT_STATUS 0x04 (alert) — or forwards a hardware
    pre-latched snapshot if configured (≤ 5 cycles to tvalid)
  → axis_injector commits: tuser=2'b11, tid = tile id
  → CDC into delta_clk; routed to HIVE-C (tdest=17) or control network
    urgent slot, per runtime configuration
```

13.4 Error Path

Parity/protocol-violation strobes set sticky `error_flags` and interrupt the core immediately; the core emits status type `0x08` and, per policy, may assert the local injection-halt line (its own aperture only — it cannot silence the compute element, which is Delta's path).

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 The Context Switch Sequence, Cycle by Cycle

The config_manager FSM (§2.5) driven by HIVE-C firmware. Typical budget: 10-20 cycles; never more than 25.

```
Cycle  Action
  1    switch_controller receives CONFIG_PREPARE (AXIL write from
firmware).
  2    Prepare broadcast issued on the control network; SERVs stop
local
       injection and (if in data path) assert quiesce request to
routers.
 3-8   Drain: routers flush in-flight packets (5-10 cycles).
switch_controller
       polls STATUS.active_packets for all participating tiles.
  9    All-zero confirmed (or hardware interlock satisfied — see §2.5
open item).
 10    SET_ROUTING_MODE PKT_CONFIG assembled (payload = context id,
       tile mask = all); injected on the HIVE-C config path.
 11    Broadcast consumed by every router in the same cycle (atomic
point:
       MODE_SELECT registers update simultaneously).
 12    config_registers.switch_status = ACTIVE_CONTEXT.
 13    CONFIG_ACTIVATE broadcast; routers resume under the new
routing.
14-20  Margin for slower drain; beyond 25 cycles the FSM raises
       switch-timeout sticky error and requires firmware rollback (HAS
§5.4).
```

Why the atomic point works: every router updates `MODE_SELECT` on the same broadcast beat. There is no skew window in which different routers hold different contexts — the switch is a single point in time (cycle 11 above), not an interval. This is the property the formal inventory asserts as “context switch atomicity.”

Quiesce verification: the FSM and firmware read the aggregated IDLE signal through the PeakRDL config register (§2.5) — all participating routers report idle before `SET_ROUTING_MODE` issues. The check is a readable status, not a hardware refusal: correct switching is firmware discipline, with a hard interlock left as optional future hardening.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 Reset and Boot Behavior

From `system_rst_n` release to a supervising HIVE system.

```
Phase  Action
  A    Staged deassertion (§1.3): host → HIVE-C → Delta → SERVs.
  B    HIVE-C fetches first instructions; firmware init: AXIL windows,
       interrupt enables, injection queue clear.
  C    Firmware loads Context 0 default (systolic) into the config
space;
       config_manager reports ACTIVE_CONTEXT = 0.
  D    SERVs release, run monitor firmware init, begin tap sampling
and
       periodic reporting.
  E    System-ready after ~100 cycles of the slowest clock (serv_clk).
       host_if accepts commands; CDA injection enabled.
```

Boot invariants: - No CDA packet may enter Delta before phase C completes (injection queue held in reset-clear until firmware enables it — a formal-friendly one-bit condition). - Routers must be in Context 0 from first ingress; a router that sees a packet while `MODE_SELECT` is undefined raises `ERROR_FLAGS` (protocol violation path, HAS §4.2). - SERVs that have not finished init must not block the control network uplink (their arbiter slot is granted and frames are simply empty/NAK — fairness holds even during boot).

Soft-reset behavior mirrors the staged sequence for the affected domain only; a SERV soft reset is invisible to Delta traffic (its tap is passive), and a Delta soft reset drains through the config_manager quiesce path rather than raw reset assertion wherever in-flight packets exist.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Verification Hooks and Observability

16.1 Observation Bus

Every block drives a fixed slice of a flat observation record (`hive_obs_pkg`), exported through `hive_top` in `hive_c_clk` domain. Slices include: injection queue occupancy/stall, descriptor_gen doorbell events, control-network grant sequence and per-slave service intervals, trigger_logic raw lines, switch_controller FSM state, and CDC FIFO full/empty flags. The repo's Ocelus monitor system attaches to this bus for protocol and coverage collection in system simulations — no internal signals are probed ad hoc.

16.2 Protocol Checkers (immediate assertions)

Checker inventory

Check	Where
tuser/tdest legal combinations (HAS §2 validation rules)	Delta ingress of each HIVE stream
No CDA before boot phase C	descriptor_gen output
Descriptor order preserved	CDC FIFO output vs. enqueue
Broadcast simultaneity (same cycle all routers)	switch_controller observation
TID integrity (tid == tile_id on CE PKT_DATA)	Delta NI (owned by Delta; HIVE TB asserts it)
Quiesce-before-switch	switch_controller (with interlock)

16.3 Formal Properties

The HAS §7 formal inventory maps to properties owned here:

- **Router deadlock freedom** — Delta-owned; HIVE testbenches assume it.
- **Descriptor FIFO ordering** — asserted on the injection queue + CDC FIFO chain.
- **Context switch atomicity** — no packet observation between `SET_ROUTING_MODE` and `CONFIG_ACTIVATE` on any router tap.
- **TID integrity** — cross-asserted with Delta NI.
- **Quiesce completeness** — `active_packets == 0` on all participating routers before the atomic point.

16.4 Testbench Structure (cocotb, rds-dv framework)

- BFMs: AXIS masters/slaves from `cocotb-framework` for the Delta-facing streams; AXIL BFM for the register window; a lightweight control-network frame BFM for the star.
- Reference: firmware-level golden models for descriptor generation and scheduling; spike is not applicable (no ISA verification — the cores are imported and verified by their own suites).
- Levels: unit per FUB (HAS §7 example), integration per block pair, system per HAS Chapter 7. Coverage targets: 100% line, 95%+ branch; functional coverage points derive from this book's behavior chapters (one covergroup per §4 sequence's decision points).

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Parameters, Defines, and Filelists

17.1 Hardware Parameters (includes/hive_config.svh)

```
// System shape
parameter int MESH_SIZE = 4;           // NxN mesh (SERV_COUNT must
equal MESH_SIZE^2)
parameter int SERV_COUNT = 16;         // number of SERV monitors
// Delta-shared fabric parameters (must match Delta's configuration)
parameter int ROUTER_BUFFER_DEPTH = 8; // flits per input VC
parameter int NUM_VIRTUAL_CHANNELS = 2;
parameter int FLIT_WIDTH = 128;        // bits
// Configuration manager
parameter int NUM_CONTEXTS = 4;        // routing configurations
// Monitoring
parameter int SERV_MEM_SIZE = 2048;    // bytes per SERV (I + D)
parameter int CONGESTION_THRESHOLD = 6;// trigger at 75% of buffer
depth
parameter int STATS_REPORT_INTERVAL = 1000; // cycles
// Performance knobs
parameter bit ENABLE_PIPELINE_BYPASS = 1;
parameter bit ENABLE_ADAPTIVE_ROUTING = 0;  // future: load balancing
```

: Hardware parameter surface

Rules: `SERV_COUNT == MESH_SIZE**2` is a static assertion in `hive_top`. Delta-shared parameters must be passed down identically to the Delta instantiation (one config include at system top, not two).

17.2 Software Knobs (sw/common/hive_config.h)

```
#define MAX_INFLIGHT_DESCRIPTORS  8
#define PRIORITY_LEVELS           4
#define AGING_THRESHOLD           1000  // cycles
#define DESCRIPTOR_BATCH_SIZE     4
#define STATS_AGGREGATION_PERIOD  10000 // cycles
```

17.3 Planned Filelist Layout

```
hive-serv/
├── rtl/
│   ├── includes/            # hive_config.svh + packages (above)
│   ├── top/hive_top.sv
│   ├── hive_c_controller/
│   ├── serv_monitor/
│   ├── control_network/
│   ├── config_manager/
│   └── common/
```

```
├── dv/
│   ├── testplans/          # arrive with RTL (kestrel testplan
convention)
│   └── tests/              # cocotb, one directory per FUB +
integration
├── models/                 # SimPy performance models (HAS ch.6
targets)
└── sw/
    ├── hive_c_fw/          # main.c, rapids_mgr.c, ... (HAS ch.5)
    └── serv_fw/            # monitor image (hand-optimized assembly)
```

: Planned repository layout

17.4 Scaling Knobs

`MESH_SIZE`/`SERV_COUNT` scale the PoC (4×4) toward the Kintex-7 8×8 expansion (HAS §7): RTL is generate-based over `SERV_COUNT`, and the control-network arbiter is a generic N-way. `NUM_CONTEXTS` grows to 8 in the expansion; `context_memory` sizing derives from it.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

18 HIVE Hardware Architecture Specification Index

Hierarchical Intelligent Vector Environment

Version: 0.2 **Date:** 2026-10-13 **Purpose:** System-level hardware architecture specification for the HIVE distributed control and monitoring subsystem (1 VexRiscv master + 16 SERV monitors) coordinating RAPIDS and Delta.

18.1 Document Organization

Note: All chapters linked below for automated document generation.

18.1.1 Front Matter

- Document Information

18.1.2 Chapter 1: Introduction

- Purpose and Scope

18.1.3 Chapter 2: System Overview

- System Overview and Context

18.1.4 Chapter 3: Subsystem Architecture

- Subsystem Architecture - the four blocks, inter-block communication, resource budget

18.1.5 Chapter 4: Interfaces

- Top-Level Interfaces
- Packet Formats and Routing Rules
- Memory Map and Register Spaces

18.1.6 Chapter 5: Programming Model

- HIVE-C Firmware Architecture
- Descriptor Scheduling
- SERV Monitor Programming
- Network Reconfiguration Workflow

18.1.7 Chapter 6: Performance

- Performance Characteristics

18.1.8 Chapter 7: Integration and Verification

- Integration and Verification

18.2 Related Documentation

- **HIVE MAS** - Micro-Architecture Specification (block-level implementation intent)
- **HIVE PRD** - Product requirements and use cases
- **HIVE CLAUDE** - AI assistance guide for this subsystem
- **RAPIDS Beats HAS** - the DMA engine HIVE-C controls
- **Delta HAS** - the network fabric HIVE configures
- **RISC-V Falcon Suite README** - suite overview and hive-serv placement beside the ladder

---

Last Updated: 2026-10-13 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Product Requirements Document (PRD)

19.1 HIVE - Hierarchical Intelligent Vector Environment

Version: 0.2 **Date:** 2026-10-12 **Status:** Specification phase — HAS/MAS v0.2 published; no RTL yet
Owner: RTL Design Sherpa Project **Parent Document:** `/PRD.md`

19.2 1. Executive Summary

The Hierarchical Intelligent Vector Environment (HIVE) is a distributed control and monitoring subsystem that coordinates the RAPIDS DMA engine and Delta Network. It demonstrates hierarchical RISC-V processor architecture, distributed monitoring, and dynamic network reconfiguration.

19.2.1 1.1 Quick Stats

- **Architecture:** 4 major blocks (HIVE-C, SERV monitors, Control network, Config manager)
- **Processors:** 1 VexRiscv (master) + 16 SERV (monitors) = 17 RISC-V cores
- **Interfaces:** Control Network (internal), AXIS (to Delta Network)
- **Resource Budget:** 6,100 LUTs HIVE control plane (14,100 including the Delta NoC), 30 BRAM on NexysA7 100T
- **Status:** Specification phase; HAS/MAS v0.2 published 2026-10-12

19.2.2 1.2 Subsystem Goals

- **Primary:** Demonstrate hierarchical control system design patterns
- **Secondary:** Enable dynamic network reconfiguration and distributed monitoring
- **Tertiary:** Educational reference for RISC-V integration and NoC control

---

19.3 2. Documentation Structure

This PRD provides a high-level overview. **Detailed specifications are maintained separately:**

19.3.1 HIVE Hardware Architecture Specification (HAS) — external contract

Location: `projects/components/riscv-ip/hive-serv/docs/hive_has/`

- **Index** - HAS structure

- System overview, subsystem architecture (4 blocks), interfaces & packet formats, programming model, performance targets, integration & verification

19.3.2 HIVE Micro-Architecture Specification (MAS) — implementation intent

Location: `projects/components/riscv-ip/hive-serv/docs/hive_mas/`

- **Index** - MAS structure
- Block-level module trees, clocking/reset, internal interfaces, cycle-level behavior of the key operations, verification hooks, parameters/filelists

19.3.3 Other Documentation

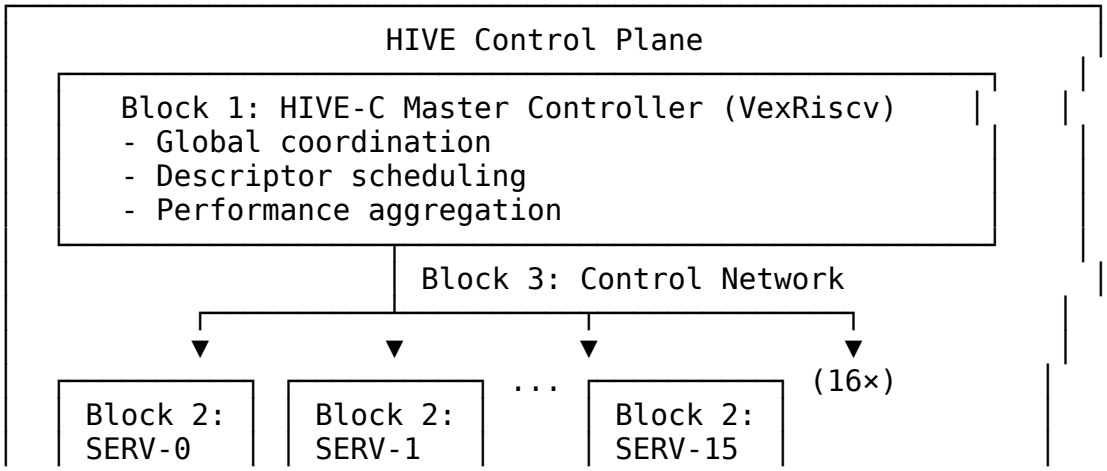
- **README** - Quick start and integration guide (to be created)
- **CLAUDE** - AI assistance guide for this subsystem
- **Tasks** - none tracked. The vault area was deleted 2026-09-27: hive has no .sv, no tests and no filelists, so its 24 open items described work that had not begun. They are in git history at `vault/Tasks/projects/components/riscv-ip/hive-serv/`.

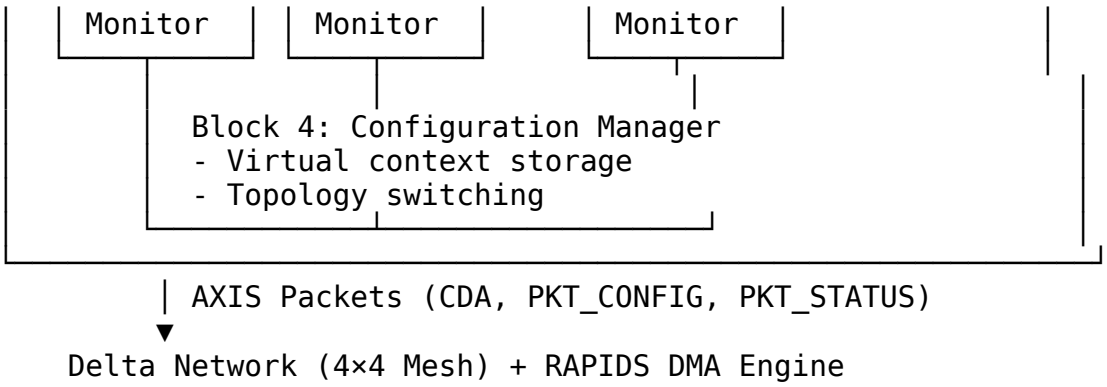
  History note: the earlier `docs/hive_spec/` + `docs/hive_specification.md` drafts (v0.3) were migrated into the HAS/MAS and retired 2026-10-12; they remain in git history.

19.4 3. Architecture Overview

19.4.1 3.1 Top-Level Block Diagram

HIVE System Architecture





See: `docs/hive_has/ch02_system_overview/01_system_overview.md` and `docs/hive_has/ch03_architecture/01_subsystem_architecture.md` for the architecture; `docs/hive_mas/` for block internals.

19.4.2 3.2 Resource Budget (NexysA7 100T)

FPGA Target: Xilinx Artix-7 XC7A100T-1CSG324C

Component	LUTs	FFs	DSPs	BRAM (36Kb)	Notes
Block 1: HIVE-C Controller	1,400	900	0	8	VexRiscv “Small” config
Block 2: SERV Monitors (16×)	2,000	2,624	0	16	125 LUTs per SERV
Block 3: Control Network	1,200	800	0	4	HIVE-C ↔️ SERV comm
Block 4: Configuration Manager	1,500	1,200	0	2	Virtual contexts
HIVE control plane subtotal	**6,100**	**5,524**	**0**	**30**	9.6% LUT, 22% BRAM
Delta Network 4×4 mesh (16 routers)	8,000	6,000	0	32	Delta-owned; see Delta HAS
HIVE + NoC total	**14,100**	**11,524**	**0**	**62**	22% LUT, 46%

Component	LUTs	FFs	DSPs	BRAM (36Kb)	Notes
					BRAM
Available for Compute/RAPIDS	49,300	115,276	240	73	Remaining resources
Total Budget	**63,400**	**126,800**	**240**	**135**	100%

> **Correction note (2026-10-12):** earlier tables presented 14,100 LUTs as the “HIVE subtotal” while omitting the Delta NoC row; the NoC’s 8,000 LUTs were silently included. The table now separates the HIVE control plane from the Delta NoC. The 22%-LUT / 46%-BRAM occupancy figures refer to the combined total.

Key Design Constraints: - HIVE consumes **zero DSP blocks** (all 240 DSPs available for compute) - HIVE’s own memory footprint is 30 BRAM blocks (22%); combined with the NoC it is 62 (46%) - HIVE uses **9.6% of LUTs** on its own (22% combined with the NoC) - Leaves **78% of logic resources** for RAPIDS and Delta Network compute elements

---

19.5 4. Key Features

19.5.1 4.1 Hierarchical RISC-V Architecture

Feature	Status	Description
VexRiscv master (HIVE-C)	note	RV32IM, 5-stage pipeline, 32KB I-mem + 32KB D-mem
SERV monitors (16×)	note	RV32I bit-serial, 2KB I-mem + 2KB D-mem each
Control network	note	Star topology, round-robin arbitration
Independent firmware	note	Separate binaries for HIVE-C and SERV

19.5.2 4.2 Distributed Monitoring

Feature	Status	Description
Per-tile traffic counters	note	Packet counts per direction (N/S/E/W)

Feature	Status	Description
Buffer occupancy tracking	note	Real-time FIFO fill levels
Congestion detection	note	Programmable thresholds, immediate alerts
Error detection	note	Parity errors, protocol violations
Periodic reporting	note	Configurable interval (default 1000 cycles)

19.5.3 4.3 Network Reconfiguration

Feature	Status	Description
Virtual configuration contexts	note	4 pre-loaded routing modes
Atomic context switching	note	Deterministic latency (<25 cycles)
Quiesce protocol	note	Drain in-flight packets before switch
Broadcast configuration	note	PKT_CONFIG packets to all tiles

19.5.4 4.4 RAPIDS Integration

Feature	Status	Description
Inband descriptor injection	note	CDA packets via Delta Network to RAPIDS
Low-latency delivery	note	~10-20 cycles vs. 100+ for memory-mapped
AXIS master interface	note	Direct injection into Delta Network
Completion tracking	note	PKT_STATUS packets from RAPIDS

19.6 5. Interfaces

19.6.1 5.1 External Interfaces

Interface	Type	Width	Purpose
Host Interface	Slave	32-bit	UART/AXI4-Lite for host commands
AXIS (HIVE-C TX)	Master	256-bit	CDA packet injection to RAPIDS
AXIS (SERV TX, 16×)	Master	128-bit	PKT_CONFIG, PKT_STATUS injection
AXIS (HIVE-C RX)	Slave	128-bit	PKT_STATUS from network
Control Network	Internal	32-bit	HIVE-C ↔️ SERV communication

See: `docs/hive_has/ch04_interfaces/` for the complete interface, packet-format, and register-map specifications.

19.6.2 5.2 Virtual Tile Mapping

Virtual Tile	Entity	Connection
Tile 16	RAPIDS DMA	Edge-attached to the 4×4 mesh
Tile 17	HIVE-C Controller	Edge-attached to the 4×4 mesh

19.7 6. Use Cases

19.7.1 6.1 Descriptor Scheduling

Scenario: HIVE-C receives high-level transfer command from host

Flow: 1. Host sends command via UART/AXI4-Lite to HIVE-C 2. HIVE-C generates 256-bit CDA descriptor 3. HIVE-C injects CDA packet via AXIS master (TUSER=2'b01, TDEST=16) 4. Delta Network routes to RAPIDS (Virtual Tile 16) 5. RAPIDS descriptor engine processes 6. Completion reported via PKT_STATUS to HIVE-C

19.7.2 6.2 Congestion Management

Scenario: SERV monitor detects buffer congestion on tile 7

Flow: 1. SERV-7 monitors router buffer occupancy 2. Threshold exceeded (buffer_occupancy > cfg_threshold) 3. SERV-7 generates PKT_STATUS alert packet 4. Alert routed to HIVE-C (Virtual Tile 17) 5. HIVE-C aggregates alerts, initiates mitigation 6. HIVE-C may trigger network reconfiguration

19.7.3 6.3 Network Reconfiguration

Scenario: Switch from packet-switched mesh to tree reduction mode

Flow: 1. HIVE-C issues CONFIG_PREPARE command via control network 2. All SERV monitors flush in-flight packets 3. Routers enter quiescent state 4. HIVE-C broadcasts SET_ROUTING_MODE (PKT_CONFIG, context=2) 5. All tiles simultaneously update routing mode 6. HIVE-C issues CONFIG_ACTIVATE command 7. Network resumes with tree reduction routing

---

19.8 7. Test Coverage

19.8.1 7.1 Current Status

Overall: Specification phase - implementation pending

Component	Spec Status	Implementation Status
HIVE-C Controller	HAS/MAS v0.2	Pending
SERV Monitors	HAS/MAS v0.2	Pending
Control Network	HAS/MAS v0.2	Pending
Configuration Manager	HAS/MAS v0.2	Pending
Integration	HAS/MAS v0.2	Pending

19.9 8. Performance Targets

19.9.1 8.1 Latency Targets

- **Descriptor Injection:** < 30 cycles (HIVE-C → RAPIDS)
- **Command Response:** < 1 μs @ 100 MHz
- **Monitoring Aggregation:** < 100 cycles for all 16 SERV status reads
- **Context Switch:** < 25 cycles (including quiesce)

19.9.2 8.2 Monitoring Overhead

- **SERV Monitor Overhead:** < 5% of compute time
- **Congestion Detection Latency:** < 10 cycles
- **Periodic Reporting Interval:** 1000 cycles (configurable)

See: `docs/hive_has/ch06_performance/01_performance.md` for the full target set and the tradeoff tables behind them.

19.10 9. Integration Guidelines

19.10.1 9.1 HIVE + RAPIDS + Delta Integration

```
// Top-level integration example (architectural port names)
module system_top (
    input  logic        clk_100mhz,
    input  logic        rst_n,
    // Host interface
    input  logic [31:0] host_cmd,
    output logic [31:0] host_resp,
    // External memory (for RAPIDS)
    // ... AXI4 memory interface ...
);

    // HIVE instance
    hive_top u_hive (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .host_cmd       (host_cmd),
        .host_resp      (host_resp),
        .axis_cda_*     (hive_to_delta_cda),
        .axis_status_*  (delta_to_hive_status)
    );

    // Delta Network instance
    delta_network_4x4 u_delta (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .tile16_*       (rapids_interface),  // RAPIDS virtual tile
        .tile17_*       (hive_interface)     // HIVE-C virtual tile
    );

    // RAPIDS instance
    rapids_top u_rapids (
        .aclk           (clk_100mhz),
```

```
        .aresetn        (rst_n),
        .axis_cda_*     (rapids_interface),  // From Delta
        .m_axi_*        (memory_interface)   // To external memory
    );

endmodule
```

See: `docs/hive_has/ch07_integration/01_integration_and_verification.md` for the full integration and verification strategy.

19.11 10. Development Status

19.11.1 10.1 Current Phase

Phase: Specification (HAS/MAS v0.2 complete, 2026-10-12)

- Hardware Architecture Specification: complete (7 chapters)
- Micro-Architecture Specification: complete (6 chapters)
- RTL: pending (roadmap in the HAS, Chapter 7)

See: nothing tracked – the vault area was deleted 2026-09-27 (unstarted); the former items are in git history.

19.11.2 10.2 Roadmap

Near-Term (Q4 2025): - HIVE-C RTL implementation (Phase 1, per HAS ch.7 roadmap) - SERV monitor wrapper implementation - Control network implementation

Long-Term (2026+): - Configuration manager implementation - HIVE-C firmware development - SERV firmware development - Integration with RAPIDS and Delta - FPGA deployment and testing

---

19.12 11. Educational Value

HIVE demonstrates: - Hierarchical processor architecture (master + agents) - Distributed monitoring and control - RISC-V processor integration (VexRiscv, SERV) - Inband control packet injection - Dynamic network reconfiguration - Resource budget management - Control network design patterns

Target Audience: - Advanced RTL designers - Network-on-chip architects - Embedded systems engineers - RISC-V integration specialists

---

19.13 13. Documentation Generation

19.13.1 13.1 Generating PDF/DOCX from the Specifications

Tool: `bin/md_to_docx.py` (repo root) via the wrapper scripts:

```
cd projects/components/riscv-ip/hive-serv/docs
./generate_has_pdf.sh            # HIVE_HAS_v0.2.docx / .pdf
./generate_mas_pdf.sh            # HIVE_MAS_v0.2.docx / .pdf
# Optional revision override:
./generate_has_pdf.sh --rev 0.2
```

The scripts check dependencies (python-docx, PyYAML, pandoc, LibreOffice), then invoke `md_to_docx.py` with the book's index, styles yaml, title page, ToC, and page breaks. Built books are checked in alongside their sources.

Requirements: Python 3.6+, pandoc, LibreOffice (PDF export), and the repo's `bin/md_to_docx.py`.

19.14 14. References

19.14.1 14.1 Internal Documentation

- **HIVE HAS:** `docs/hive_has/` ← **Primary external-contract reference**
- **HIVE MAS:** `docs/hive_mas/` ← **Primary implementation-intent reference**
- **Master PRD:** `/PRD.md`
- **Repository Guide:** `/CLAUDE.md`

19.14.2 14.2 Related Subsystems

- **RAPIDS:** `projects/components/dma-ip/rapids/` - DMA engine controlled by HIVE-C
- **Delta Network:** `projects/components/fabric-gen-ip/delta/` - AXIS fabric generator (the HIVE fabric; owner: same fabric as the archived NoC draft)
- **AMBA/Ocelus:** `rtl/amba/ocelus/` - Monitor infrastructure for observability
- **Common:** `rtl/common/` - Building blocks

19.14.3 14.3 External References

- **VexRiscv:** https://github.com/SpinalHDL/VexRiscv
- **SERV:** https://github.com/olofk/serv

- **RISC-V ISA:** https://riscv.org/technical/specifications/
- **AXI4-Stream:** ARM IHI 0051A

Document Version: 0.2 **Last Updated:** 2026-10-12 **Review Cycle:** Monthly during active development **Owner:** RTL Design Sherpa Project

---

19.15 Navigation

- **← Back to Root:** `/PRD.md`
- **Hardware Architecture Specification:** `docs/hive_has/hive_has_index.md`
- **Micro-Architecture Specification:** `docs/hive_mas/hive_mas_index.md`
- **Quick Start:** `README.md` (to be created)
- **AI Guidance:** `CLAUDE.md`
- **Tasks:** none tracked (vault area deleted 2026-09-27, unstarted)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 RAPIDS Beats Architecture MAS

Version: 0.7 **Date:** 2026-07-13 **Purpose:** Module Architecture Specification for RAPIDS Phase 1 "Beats" Architecture

---

20.1 Document Organization

Note: All chapters linked below for automated document generation.

20.1.1 Front Matter

- Document Information

20.1.2 Chapter 1: Overview

- Architecture Overview
- Top-Level Port List
- Clocks and Reset

20.1.3 Chapter 2: FUB (Functional Unit Blocks)

Control Path: - Scheduler - Descriptor Engine

AXI Engines: - AXI Read Engine - AXI Write Engine

Flow Control: - Beats Alloc Control - Beats Drain Control - Beats Latency Bridge

Control Engines: - Control-Read Engine - Control-Write Engine

20.1.4 Chapter 3: Macro (Integration Blocks)

Scheduler Integration: - Beats Scheduler Group - Beats Scheduler Group Array

Sink Data Path (Network to Memory): - Sink Data Path - Sink Data Path AXIS - Sink SRAM Controller

Source Data Path (Memory to Network): - Source Data Path - Source Data Path AXIS - Source SRAM Controller

Top-Level Integration: - RAPIDS Core Beats - RAPIDS Registers - RAPIDS Config Block - RAPIDS Beats Top

20.1.5 Chapter 4: Interfaces

- Interface Overview

- AXI4 Interface Specification
- AXIS Interface Specification
- MonBus Interface Specification

20.2 Quick Reference

20.2.1 FUB Modules (fub_beats/)

FUB Module Summary

Module	File	Purpose	Status
scheduler_beats	`scheduler_beats.sv`	Transfer coordinator, descriptor processing	Implemented
descriptor_engine_beats	`descriptor_engine_beats.sv`	Descriptor fetch/parse (256-bit)	Implemented
axi_read_engine_beats	`axi_read_engine_beats.sv`	AXI read master, streaming pipeline	Implemented
axi_write_engine_beats	`axi_write_engine_beats.sv`	AXI write master, streaming pipeline	Implemented
alloc_ctrl_beats	`alloc_ctrl_beats.sv`	Space allocation tracking (virtual FIFO); kept and tested, no longer in the SRAM path since `bdf4e0dff`	Implemented
drain_ctrl_beats	`drain_ctrl_beats.sv`	Data availability tracking (virtual FIFO); kept and tested, no longer in the SRAM path	Implemented
latency_bridge_beats	`latency_bridge_beats.sv`	Registered-FIFO to valid/ready skid bridge; kept and tested, no longer in the SRAM path	Implemented
ctrlrd_engine	`ctrlrd_engin`	Control-read	Implemented

Module	File	Purpose	Status
	e.sv	consumer gate (poll-until-match, retry budget)	
ctrlwr_engine	ctrlwr_engine.sv	Control-write producer doorbell (single-beat write)	Implemented

20.2.2 Macro Modules (macro_beats/)

Macro Module Summary

Module	File	Purpose	Status
scheduler_group_beats	scheduler_group_beats.sv	Scheduler + Descriptor Engine wrapper	Implemented
scheduler_group_array_beats	scheduler_group_array_beats.sv	8-channel scheduler array with arbitration	Implemented
snk_data_path_beats	snk_data_path_beats.sv	Network-to-memory path integration	Implemented
snk_data_path_axis_beats	snk_data_path_axis_beats.sv	AXIS variant of sink path	Implemented
src_data_path_beats	src_data_path_beats.sv	Memory-to-network path integration	Implemented
src_data_path_axis_beats	src_data_path_axis_beats.sv	AXIS variant of source path	Implemented
snk_sram_controller_beats	snk_sram_controller_beats.sv	Sink SRAM: naming wrapper around STREAM's sram_controller (one instance, per-channel FIFOs inside it)	Implemented
src_sram_controller_beats	src_sram_controller_beats.sv	Source SRAM: naming wrapper around STREAM's	Implemented

Module	File	Purpose	Status
		`sram_controller`	
rapids_src_beats / rapids_snk_beats	`rapids_src_beats.sv`, `rapids_snk_beats.sv`	One half each: scheduler array + data path + monbus merge	Implemented
rapids_core_beats	`rapids_core_beats.sv`	Both halves + the core monbus arbiter	Implemented
rapids_regs	`regs/generated/rtl/rapids_regs.sv` (generated from `rtl/macro_beats/rapids_regs.rdl`)	PeakRDL register block (base + MON regfile @ 0x1000)	Implemented
rapids_config_block	`rapids_config_block.sv`	Maps register `hwif_out` to core/monitor `cfg_*`	Implemented
rapids_beats_top	`rapids_beats_top.sv`	Top-level: APB slave, kickoff, monitors, MonBus AXI-Lite group	Implemented

20.3 Architecture Comparison: RAPIDS vs STREAM

The “beats” architecture is a Phase 1 implementation of RAPIDS that shares concepts with STREAM but targets network-to-memory (and vice versa) transfers rather than memory-to-memory.

RAPIDS Beats vs STREAM Comparison

Feature	STREAM	RAPIDS Beats
Primary Use	Memory-to-memory DMA	Network-to-memory accelerator
Data Paths	Single bidirectional	Separate sink/source paths

Feature	STREAM	RAPIDS Beats
Network Interface	None	AXIS master/slave
SRAM Buffering	Shared buffer	Separate sink/source buffers
Descriptor Format	256-bit	256-bit (compatible)
Channel Count	8	8
Flow Control	alloc_ctrl/ drain_ctrl	STREAM `stream_alloc_ctrl`/`stream_drain_ctrl` inside the shared `sram_controller` (`alloc_ctrl_beats`/`drain_ctrl_beats` kept, out of the path)

20.4 Clock and Reset Summary

20.4.1 Clock Domains

Clock Domains

Clock	Frequency	Usage
`aclk`	100-500 MHz	Primary - all RAPIDS logic, AXI/AXIS interfaces

20.4.2 Reset Signals

Reset Signals

Reset	Polarity	Type	Usage
`aresetn`	Active-low	Async assert, sync deassert	Primary - all RAPIDS logic

See: Clocks and Reset for complete timing specifications

20.5 Interface Summary

20.5.1 External Interfaces

External Interfaces

Interface	Type	Width	Purpose
AXI4 (Descriptor)	Master	256-bit	Descriptor fetch
AXI4 (Sink Write)	Master	512-bit (param)	Sink data write to memory
AXI4 (Source Read)	Master	512-bit (param)	Source data read from memory
AXIS (Sink)	Slave	512-bit (param)	Network data ingress
AXIS (Source)	Master	512-bit (param)	Network data egress
MonBus	Master	64-bit	Monitor packet output

20.5.2 Internal Buses

Internal Buses

Interface	Width	Purpose
MonBus	64-bit	Internal monitoring bus
Descriptor Bus	256-bit	Descriptor distribution
SRAM Data Bus	512-bit	SRAM read/write data

20.6 Related Documentation

- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **RAPIDS Beats HAS** - Architecture specification (the original pre-beats rapids_spec/ tree was retired)

20.7 Specification Conventions

20.7.1 Signal Naming

- **Clock:** `aclk`
- **Reset:** `aresetn` (active-low)
- **Valid/Ready:** Standard AXI/AXIS handshake
- **Registers:** `r_` prefix (e.g., `r_state`, `r_counter`)
- **Wires:** `w_` prefix (e.g., `w_next_state`, `w_grant`)

20.7.2 Parameter Naming

- **Uppercase:** `NUM_CHANNELS`, `DATA_WIDTH`, `ADDR_WIDTH`
- **Derived:** `CHAN_WIDTH = $clog2(NUM_CHANNELS)`

20.7.3 Figure and Table Numbering

- **Figures:** `Figure X.Y.Z: Title` where X=chapter, Y=section, Z=figure number
- **Tables:** Caption line `: Table Title` after table markdown

Last Updated: 2026-07-13 **Maintained By:** RAPIDS Architecture Team

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 Delta Micro-Architecture Specification Index

Component: Delta (AXI4-Stream Crossbar Generator) **Version:** 1.0 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for the generated Delta RTL

21.1 Overview

Delta is a Python generator that emits a single self-contained SystemVerilog module per configuration — a flat M×N AXI4-Stream crossbar routing `s_axis_*` masters to `m_axis_*` slaves. This is the internals document: the one you open when you need to know what the generated RTL actually does, block by block, rather than what the high-level spec says.

21.1.1 Key Features

- **Flat crossbar only:** Tree generation was retired on 2026-10-12.
- **TDEST-based routing:** No address map; the destination slave is taken directly from `s_axis_tdest`.
- **Packet-atomic round-robin arbitration:** A granted master holds the slave until `TLAST`.
- **Combinational data mux:** The grant is the registered stage; payload passes through combinatorially.
- **Optional richness:** `TSTRB` byte qualifiers, and AMBA 5 `TWAKEUP` with the IHI 0051B ch. 5 odd-parity check signals (`TDATACHK` … `TWAKEUPCHK`), checked at every port with sticky error flags (default off).
- **Optional counters:** Per-master and per-slave packet counters (disable with `--no-counters`).
- **Formal assertions:** `ifdef FORMAL` checks for one-hot grants and in-range TDEST.

21.1.2 Companion Specifications

- **Delta HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

21.2 Navigation

21.2.1 Front Matter

- Document Information

21.2.2 Chapter 1: Introduction

- Overview

21.2.3 Chapter 2: Block Descriptions

- Request Decode
- Per-Slave Round-Robin Arbiter
- Crossbar Multiplexer
- TREADY Steering
- TWAKEUP Steering (AMBA5)
- Interface Check Signals (AMBA5)
- Packet Counters
- Assertions

21.2.4 Chapter 3: Cycle-Level Behavior

- Multi-Master Contention
- Packet Interleave Prevention

21.2.5 Chapter 4: Generated RTL

- Module Structure
- Parameters and Logic Implications

21.2.6 Chapter 5: Verification

- Test Strategy

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 RISC-V Falcon Suite

Five independent, clean-slate RISC-V cores whose primary product is **education**, in the same spirit as the pumice/scoria/andesite memory-controller line. Each rung is a complete, verified, documented design small enough to hold in your head; the rungs ascend in microarchitectural complexity so the *diff between neighbors is the lesson*. Rungs do not share RTL — each core is the cleanest possible example of its level, not the cheapest to produce. Educational benefit over IP; no rung chases performance or benchmarks.

Design spec (source of truth for scope, ISA, verification, packaging): `docs/superpowers/specs/2026-10-06-riscv-falcon-suite-design.md`.

22.1 The ladder

Each rung carries a raptor-codename, real falcons first (species in strict size order, tracking complexity). The falcon run ends at gyrfalcon — the largest living species — so the top rung steps deliberately to mythology: garuda, the giant divine raptor, king of birds, one past the largest falcon. The codename is the RTL identifier prefix and the module/package prefix; the DIRECTORY is the compound `<codename>-<isa>` form — `kestrel-rv32i`, not `kestrel`. The bare codename is how the core is referred to in prose ("kestrel sees the whole machine in one cycle").

Directory	Codename	ISA	Microarchitecture	Status
`kestrel-rv32i/`	**kestrel**	RV32I	single-cycle	RTL complete; rv32ui battery 42/42 with spike lockstep; riscv-formal 42/42; doc book v0.1 published; board loader sim-verified
`merlin-rv32i/`	**merlin**	RV32I	5-stage in-order pipeline	Planned; structure only
`peregrine-rv32im/`	**peregrine**	RV32IM	advanced in-order: predictor, traps, L1 caches, AXI	Planned; structure only

Directory	Codename	ISA	Microarchitecture	Status
			master	
`gyrfalcon-rv32ima/`	**gyrfalcon**	RV32IM baseline; gyrfalcon-A flavor RV32IMA	out-of-order capstone: rename, ROB, reservation stations; two flavors — baseline and gyrfalcon-A (`A_EXT=1`: LR/SC + AMO atomics study)	Planned; structure only
`garuda-rv32imaf/`	**garuda**	RV32IMAF	gyrfalcon's OoO plus an FPU: FP register file + rename, fcsr precise flags, non-pipelined long-latency units	Planned; structure only (penciled in; beyond the 2026-10-06 design spec, which pins four rungs)

22.1.1 Adjacent: hive-serv

`hive-serv/` — the SERV-based compute cluster (1 VexRiscv control plane + 16 SERV monitor cores). Not a falcon-ladder rung: it *uses* RISC-V cores rather than teaching one, so it lives beside the ladder rather than on it. Re-homed from `compute-eng-ip/hive` 2026-10-10 (it was retired 2026-09-27 as an unstarted component; the re-home revives it under the falcon suite). HAS/MAS v0.2 published 2026-10-13 (`hive-serv/docs/hive_has/`, `hive-serv/docs/hive_mas/`) — no RTL yet.

22.2 Goals per rung

kestrel — see everything. Single-cycle RV32I: PC, register file, ALU, immediate generator, control as an explicit decode truth table, a plainly documented combinational-read memory contract. The hovering falcon: the whole machine is visible in one cycle — including its cost, since the critical

path *is* the machine, which is exactly the motivation for merlin. **Delivers:** the ISA (R/I/S/B/U/J formats, the full base integer set), the datapath, honest control logic.

merlin — flow it. The classic IF/ID/EX/MEM/WB pipeline on the deliberately identical RV32I ISA, so students diff *behavior*, not ISA. Forwarding from EX/MEM and MEM/WB with explicit priority, load-use interlock stall, branches resolved in EX predict-not-taken with flush. **Delivers:** pipeline registers, RAW hazards, forwarding priority, the load-use bubble, control-hazard flushes, why branch resolution stage matters.

peregrine — speed it up. Advanced in-order RV32IM: iterative multicycle multiply/divide (the structural-hazard lesson), BTB + 2-bit bimodal prediction with mispredict recovery, machine-mode CSRs with precise exceptions and CLINT-shaped interrupts, split blocking write-back L1 caches behind an AXI4 master on the repo fabric with MonBus visibility. **Delivers:** speculation and recovery, precise exceptions, the memory hierarchy, bus protocol — and ties the whole repo together (memory controllers, cache IP line, AMBA fabric).

gyrfalcon — hide everything. Out-of-order capstone in two flavors (added 2026-10-11): the baseline — peregrine's RV32IM exactly — and gyrfalcon-A (`A_EXT=1`): LR/SC reservations, store-conditional failure, and AMOs, the atomicity study — so every peregrine battery still carries in both. Dual-issue dispatch, explicit rename map + free list, 32-entry ROB with in-order commit, three reservation stations, wakeup/select over common data buses, speculative dispatch with checkpoint-restore rename rollback. **Delivers:** register renaming and WAR/WAW elimination, the ROB as an undo log, out-of-order completion with in-order retirement — the honest, complete OOO story — and the atomicity study: reservations, SC failure, AMOs, acquire/release as memory-model operations.

garuda — float it. The RV32IMAF delta on gyrfalcon's OoO: a 32-entry FP register file with its own rename, an `fcsr` whose five accrued exception flags (NV/DZ/OF/UF/NX) stay precise-exception-clean through the ROB, and the repo's ieee754 fp32 arithmetic behind the reservation stations — multi-cycle FMUL/FADD and the non-pipelined Goldschmidt FDIV / Newton-rsqrt FSQRT (9–11 cycle class) as the long-latency-unit lesson. Rounding hard-wired RNE; any other requested mode raises NV, keeping the CSR story honest without a dynamic rounding-mode datapath. **Delivers:** long-latency execution scheduling, FP architectural state, and the F-extension ISA delta (F regs, `fcsr`, FMV/FCVT/FSGNJ/FCLASS) — consuming the math library's `math_ieee754_2008_fp32_*` blocks end-to-end.

22.3 Packaging (per rung, the MC-line treatment)

- **Doc book** — *Simplified RV32I: kestrel* and successors, Memory Notes branding, built through `bin/md_to_docx.py` to DOCX/PDF, following the Simplified DFI books' precedent. Each book lives in its rung's `docs/`; the kestrel edition is published at `kestrel-rv32i/docs/simplified_rv32i/`.
- **Formal** — riscv-formal-style ISA compliance where it pays: full on kestrel, retire-interface on merlin, targeted properties (exception precision, recovery, in-order commit, deadlock freedom) on peregrine/gyrfalcon.

- **Golden model** — spike ISS lockstep in the rds-dv cocotb framework; the classic `rv32ui`/`rv32um` riscv-tests batteries as the compliance gate.
- **Drill app** — `bin/apps/core_drills` (ddr_drills architecture): pipeline stage-viewer with forwarding arrows, hazard quiz, branch-predictor drill, and a rename/ROB viewer. App design gets its own doc once merlin's RTL is fixed.

22.4 References and layout

Common primary sources (the RISC-V ISA manuals) live in `references/` and are cited by chapter, not page.

Per-rung skeleton follows the dma-ip/stream component layout convention: `rtl/`, `docs/`, `dv/` (cocotb + lockstep testbenches). Shared suite doctrine lands in `docs/` as it is written. Per-rung `PRD.md`/`CLAUDE.md` arrive with each rung's implementation plan.